Understanding Idempotence

Idempotence can be a difficult concept to grasp, and it can be equally difficult to determine that a routine or job *is* idempotent. The key is to ask yourself what will happen if the method is called more than once with the same arguments.

Here is a simplified example, where a job is adding more quantity to a product:

```
class AddMoreProductsJob
  include Sidekiq::Job
  def perform(product_id, num_new_products)
    product = Product.find(product_id)
    product.update!(
      quantity_remaining: product.quantity_remaining + num_new_products
    )
  end
end
```

If the job gets retried for some reason, even if every line of code has executed, it will have added twice the number of products as intended. That means it's not idempotent. You could make it idempotent by removing the calculation and passing in the final value:

```
class AddMoreProductsJob
  include Sidekiq::Job
  def perform(product_id, new_quantity_remaining)
    product = Product.find(product_id)
    product.update!(
      quantity_remaining: new_quantity_remaining
    )
  end
end
```

The job can be safely retried and is idempotent. However, it's less useful because the calculation for the correct value of `quantity_remaining` has to be